If we want to add interactivity, it's a simple process because we know that each rect in the legend corresponds to a two-piece array of values that we can use to test the circles in our grid. The following listing shows that function and the call to make the legend interactive.

Listing 10.14 Legend interactivity

```
d3.select("#legend").selectAll("rect").on("mouseover", legendOver);

function legendOver(d) {
    console.log(d)
    d3.selectAll("circle")
        .style("opacity", function(p) {
           if (p.size >= d.domain[0] && p.size <= d.domain[1]) {
                 return 1;
           } else {
                 return .25;
           }
        });
};
```

Notice that this function isn't defined inside our legend component. Instead, it's defined and called after the legend is created, because after it's created, our legend component is just a set of SVG elements with data bound to it like any other part of our charts. This interactivity allows us to mouseover the legend and see which circles fall in a particular range of values, as shown in figure 10.10.

Finally, before we can call our legend done, we need to add an indication of what those colored bands mean. We could call an axis component and allow that to label the bands, or we can label the break points by appending text elements for each. In our case, because the numbers provided for `d3.geo.area` are so small, we'll also need to rotate and shrink those labels quite a bit for them to fit on the page. To do this, we can add the code in listing 10.15 to our legend function in `d3.svg.legend`.

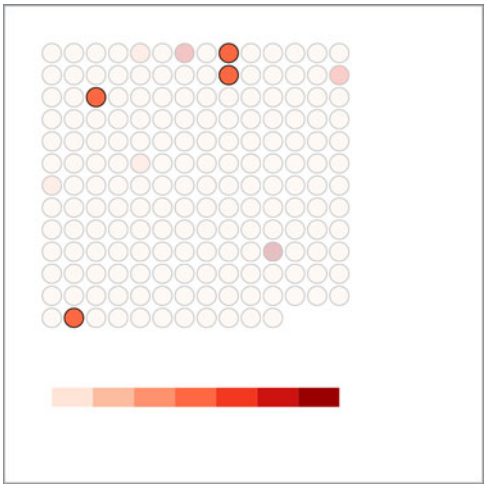


Figure 10.10 The `legendOver` behavior highlights circles falling in a particular band and deemphasizes the circles not in that band by making them transparent.